

课程笔记

[LLM Agents SP25] Multimodal Agents: Perception to Action

基于 Caiming Xiong 授课内容整理

2026-04-02



The slide features a blue background with a white circular portrait of Caiming Xiong on the left. To the right of the portrait, the text 'CAIMING XIONG' is displayed in bold, followed by 'SVP OF AI RESEARCH & APPLIED RESEARCH' and the Salesforce logo. Above the portrait, the text 'UC Berkeley Center for Responsible, Decentralized Intelligence' is visible. To the right of the portrait, the text 'Advanced Large Language Model Agents MOOC' is displayed in bold. Below this, the title 'Multimodal Agents - From Perception to Action' is shown in a large, bold font. At the bottom right, the date 'March 17th, 2025' is displayed. A decorative pattern of white dots is located in the bottom right corner.

UC Berkeley Center for Responsible,
Decentralized Intelligence

**Advanced Large Language
Model Agents MOOC**

**Multimodal Agents –
From Perception to
Action**

CAIMING XIONG
SVP OF AI RESEARCH &
APPLIED RESEARCH
salesforce

March 17th, 2025

视频作者/频道: Berkeley RDI

发布日期: 2025-03-20

视频时长: 1:28:21

视频链接: https://www.youtube.com/watch?v=n__Tim8K2IY

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 从基准饱和到多模态 Agent：问题空间为什么被重写	3
1.1 主讲脉络与核心判断	3
1.2 为什么“多模态”不再是可选项	3
1.3 本章小结	4
2 应用图谱：数字工作流如何成为 Agent 的主战场	4
2.1 任务形态：单应用操作到跨应用编排	4
2.2 Agent 能力分层：感知、规划、执行、验证	4
2.3 数字场景优先，物理场景随后	4
2.4 本章小结	5
3 Benchmark 体系升级：从网页基准到 OS-World	5
3.1 现有基准的价值与边界	5
3.2 OS-World 的设计目标	5
3.3 对比视角：为什么 OS-World 更接近生产问题	6
3.4 本章小结	6
4 OS-World 的执行机制：观察、动作与交互循环	7
4.1 观察空间设计	7
4.2 动作空间与执行接口	7
4.3 交互循环	7
4.4 工程细节：安全与可复现	8
4.5 本章小结	8
5 评估与奖励：如何把开放任务变成可训练目标	8
5.1 执行式评估（Execution-based Evaluation）	8
5.2 部分完成奖励与难度分层	9
5.3 评测体系的鲁棒性要求	9
5.4 本章小结	9
6 数据瓶颈：从人工标注到教程驱动轨迹生成	10
6.1 为什么轨迹数据是核心稀缺资源	10
6.2 教程驱动（Tutorial-driven）数据构造	10
6.3 质量控制：从“可运行”到“可学习”	10
6.4 本章小结	11
7 合成数据与 Action Calling：构建可扩展训练飞轮	11
7.1 从感知任务到动作任务的迁移	11
7.2 典型合成流程	11
7.3 合成数据的边界	12
7.4 本章小结	12

8 模型设计：统一感知、规划与执行的多阶段架构	12
8.1 现有 GUI Agent 的关键短板	12
8.2 课程提出的整合思路	12
8.3 Planner 信号的引入	13
8.4 本章小结	13
9 实验结果与可操作结论	13
9.1 结果解读：趋势比绝对数值更重要	13
9.2 落地指标建议	14
9.3 本章小结	15
10 工程落地路线图：从研究原型到生产系统	15
10.1 阶段一：可复现实验平台	15
10.2 阶段二：数据飞轮	15
10.3 阶段三：生产守护	15
10.4 本章小结	15
11 失败案例与调试手册：把不可用系统变成可迭代系统	15
11.1 失败类型分层：先分桶再修复	15
11.2 调试闭环：日志、回放、对照实验	16
11.3 高频问题实例	16
11.4 本章小结	17
12 研究议程：下一代多模态 Agent 的三个突破口	17
12.1 突破口一：统一表示层	17
12.2 突破口二：可验证长程规划	17
12.3 突破口三：安全与治理原生化	17
12.4 未来 12 个月可执行任务清单	18
12.5 本章小结	18
13 总结与延伸	18
13.1 全讲总结表	18
13.2 进一步阅读	18
13.3 延伸问题	19

1 从基准饱和到多模态 Agent：问题空间为什么被重写

1.1 主讲脉络与核心判断

本讲由 Salesforce AI Research 的 Caiming Xiong 主讲，主线非常清晰：传统文本任务基准在过去几年接近饱和，下一阶段的挑战不再是“离线答题”，而是“在真实数字环境中持续完成任务”。这一定义把研究对象从静态 NLP 任务，切换为可交互、多步骤、跨应用的 Agent 系统。

从“会回答”到“会做事”

在多模态 Agent 范式里，模型价值不只是输出一句话是否正确，而是能否稳定地完成一个真实工作流：读界面、理解状态、规划动作、执行操作、校验结果、再迭代。这个闭环才是生产环境里的真正门槛。

1.2 为什么“多模态”不再是可选项

讲者把多模态 Agent 定义为 Vision-Language-Action (VLA) 系统。原因很直接：现实工作流并不是纯文本 API，更多是 GUI、网页、桌面软件、移动端界面，甚至外设交互。Agent 只有同时具备视觉理解、语言推理和动作生成能力，才可能接管复杂流程。

多模态 Agent 的输入输出对比

- 输入从“单轮文本”变成“用户指令 + 屏幕截图 + 可访问性树 + 环境状态”；
- 输出从“自然语言答案”变成“可执行动作序列”；
- 评估从“准确率”变成“任务完成率、轨迹质量、步骤效率与鲁棒性”。

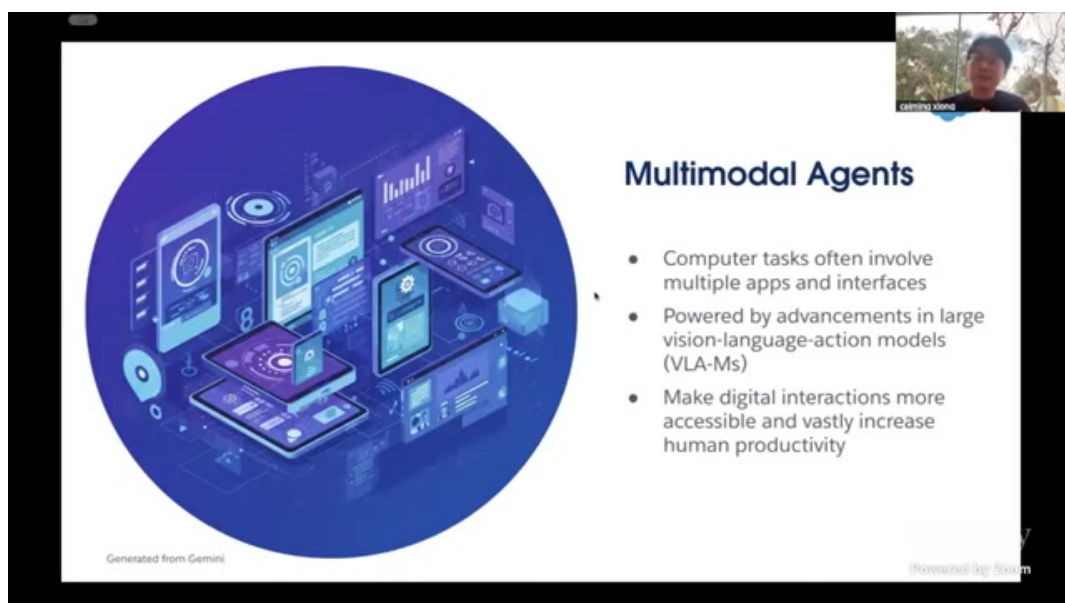


图 1: 视频约 00:01:30: 讲者展示近年基准快速饱和，并引出 Agent 评测要从离线题库转向真实环境

常见误解：把“会用工具”当成“可部署 Agent”

演示里一次成功并不代表系统可上线。生产级 Agent 关注的是长期稳定性：错误恢复、任务中断处理、跨应用状态一致性、权限边界和审计可追踪性。这些都不是单个 benchmark 分数能覆盖的。

1.3 本章小结

多模态 Agent 的研究重点已从模型“答题能力”转向“环境交互能力”。这一转变决定了后续必须同时建设三层基础：真实环境 benchmark、可扩展数据管线、可执行的模型架构。

2 应用图谱：数字 workflows 如何成为 Agent 的主战场

2.1 任务形态：单应用操作到跨应用编排

讲者强调，实际用户任务通常不是一个页面一次点击，而是跨工具流程。例如：浏览器检索信息、表格整理数据、邮件发送结果、文档归档记录。多模态 Agent 需要处理持续状态与任务上下文，而不是短期指令响应。

真实世界任务的四个特征

1. **长链路**：常常超过 20-50 步；
2. **跨界面**：浏览器、桌面、终端、Office 工具混合出现；
3. **非确定性**：页面加载、弹窗、网络波动导致状态变化；
4. **目标开放**：很多任务存在多条可行路径，不是单一标准答案。

2.2 Agent 能力分层：感知、规划、执行、验证

课程把 Agent 拆成能力栈，而不是一个黑盒模型。感知层负责理解界面与语义对象；规划层决定下一步子目标；执行层输出可执行动作；验证层判定任务是否完成并修正轨迹。这种分层能更好定位系统瓶颈。

为什么多模态 Agent 需要“过程可见性”

当任务失败时，团队必须知道问题在感知、规划还是执行。如果所有失败都被归因到“模型不够强”，就无法制定有效优化策略。可观测性是 Agent 迭代效率的核心。

2.3 数字场景优先，物理场景随后

从落地节奏看，讲者明显把数字环境（web、desktop、mobile）作为优先级更高的方向。原因是反馈闭环更快、评测脚本更容易搭建、成本更可控。物理世界 Agent 虽然重要，但数据与评测基础设施更重，迭代速度更慢。

不要过早把机器人路径与 GUI Agent 混为一谈

两者都属于 Agent，但工程约束完全不同。GUI Agent 的瓶颈在界面理解和任务编排；物理 Agent 还叠加感知误差、安全约束和执行器稳定性。混合讨论会掩盖问题本质。

2.4 本章小结

课程把应用问题定义为“跨应用、多步骤、可恢复”的长期任务。由此引申出的工程要求是：先把数字任务闭环做深，再向更复杂场景扩展。

3 Benchmark 体系升级：从网页基准到 OS-World

3.1 现有基准的价值与边界

讲者回顾了 Mind2Web、WebArena、VisualWebArena 等基准。这些工作推动了 Web Agent 研究，但它们的环境覆盖、任务类型和可扩展性仍不足以代表真实“computer use”场景，尤其是跨应用与开放式目标任务。

典型不足

- 任务分布偏网页导航，覆盖面窄；
- 许多任务存在隐式模板化路径，容易被策略过拟合；
- 对桌面应用、文件系统、跨软件流程支持不足；
- 难以承载“开发-测试-评测”一体化闭环。

3.2 OS-World 的设计目标

OS-World 被定位为更通用的真实计算机环境：在虚拟机中提供可交互桌面与应用生态，支持研究者定义任务、执行 Agent、收集轨迹、运行评测脚本，并且保留扩展空间。

OS-World 的三项关键属性

1. **真实性**：接近真实操作系统与应用行为；
2. **可扩展性**：可持续新增任务与应用组合；
3. **可评测性**：每个任务可绑定脚本化验证逻辑。

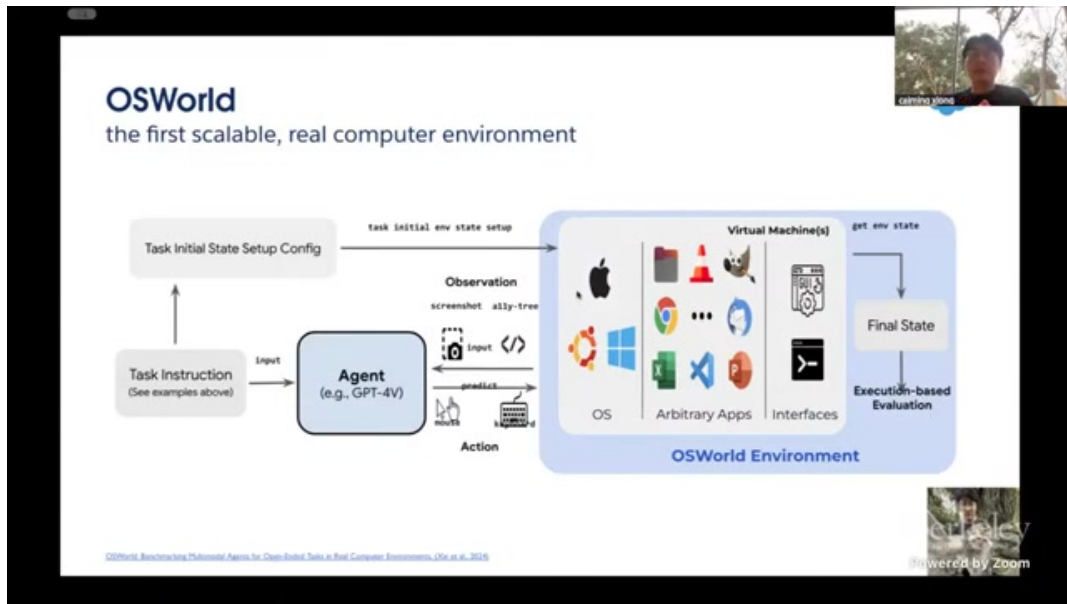


图 2: 视频约 00:10:32: OS-World 作为可扩展真实环境被提出，用于多模态 Agent 的开发与评测

3.3 对比视角：为什么 OS-World 更接近生产问题

维度	传统 Web 基准	OS-World
环境覆盖	主要网页域	网页 + 桌面应用 + 系统交互
任务形态	偏导航与表单	跨应用、多步骤、开放式流程
评测方式	页面结果导向	脚本验证 + 部分完成奖励
扩展能力	新任务构建成本较高	支持系统化新增任务与配置
工程适配	偏离线评测	开发、训练、评测一体化

表 1: OS-World 并非替代所有基准，而是补齐“真实计算机任务”的核心空缺

评测迁移风险

如果团队只在单一 benchmark 上优化，模型容易对评测脚本形成“策略投机”。课程建议在多环境、多任务族上做交叉验证，以防止 benchmark overfitting。

3.4 本章小结

OS-World 的意义不只是一个 benchmark，而是把 Agent 研究从“题库竞争”推向“环境工程”。这为后续的数据与模型设计提供了统一实验底座。

4 OS-World 的执行机制：观察、动作与交互循环

4.1 观察空间设计

课程讲解了多源观察输入：屏幕截图、可访问性树 (accessibility tree)、终端输出和自定义状态流。不同 Agent 可按需求选择视觉主导或文本主导的输入组合。

为什么截图 + 可访问性树要同时保留

截图提供整体视觉上下文，但结构化程度低；可访问性树具备对象层级与属性信息，便于精准定位和可解释操作。二者互补能显著提高动作生成稳定性。

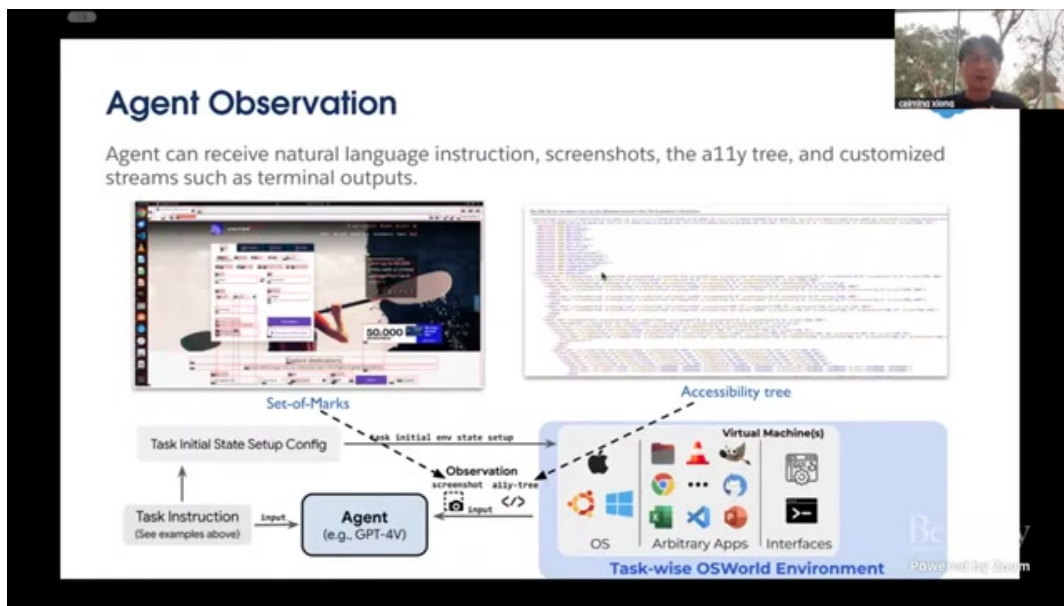


图 3: 视频约 00:16:30: 讲者展示 screenshot 与 accessibility tree 的组合观察形式

4.2 动作空间与执行接口

动作由可执行指令表达，覆盖点击、输入、滚动、快捷键与组合操作。Agent 并非直接输出“建议”，而是生成可在虚拟机中执行的行为代码。

动作格式标准化的价值

统一动作格式能同时服务三件事：训练数据对齐、执行器稳定运行、评测脚本可复现。没有标准动作层，跨模型比较和大规模数据积累都会失败。

4.3 交互循环

系统循环为“接收观察 → 规划动作 → 环境执行 → 新观察”。任务在达到目标、触发失败条件或到达最大步数时结束。这个循环把 Agent 训练目标从单步预测扩展为序列决策。

```
1 obs = env.reset(task_config)
2 for step in range(max_steps):
3     action = agent.predict(obs, instruction)
```



```
4     obs, reward, done, info = env.step(action)
5     if done:
6         break
```

Listing 1: 多模态 Agent 在 OS-World 的简化交互循环

4.4 工程细节：安全与可复现

课程强调了虚拟机隔离和任务配置文件的重要性。研究环境必须可重置、可记录、可重放，这样才能做稳定对比实验并诊断失败轨迹。

常见失败点

真实环境里最易被忽略的是“状态漂移”：缓存、登录态、弹窗与后台进程导致同任务在不同运行中行为不一致。必须通过环境快照、初始化脚本和日志追踪降低漂移影响。

4.5 本章小结

OS-World 的核心不是界面截图本身，而是“结构化观察 + 标准动作 + 可重放执行”的闭环设计。这让多模态 Agent 研究进入可工程化迭代阶段。

5 评估与奖励：如何把开放任务变成可训练目标

5.1 执行式评估 (Execution-based Evaluation)

课程采用脚本化执行评估，而非人工主观打分。任务完成后，由验证脚本检查文件、页面状态、输出内容或系统变量，给出完成度判断。

开放任务也可以做可计算评估

即使任务有多条路径，只要结果状态可验证，就能构建自动评测。关键是把“任务意图”映射为“可检查条件”，例如文件存在性、字段值、页面元素状态、命令返回值等。

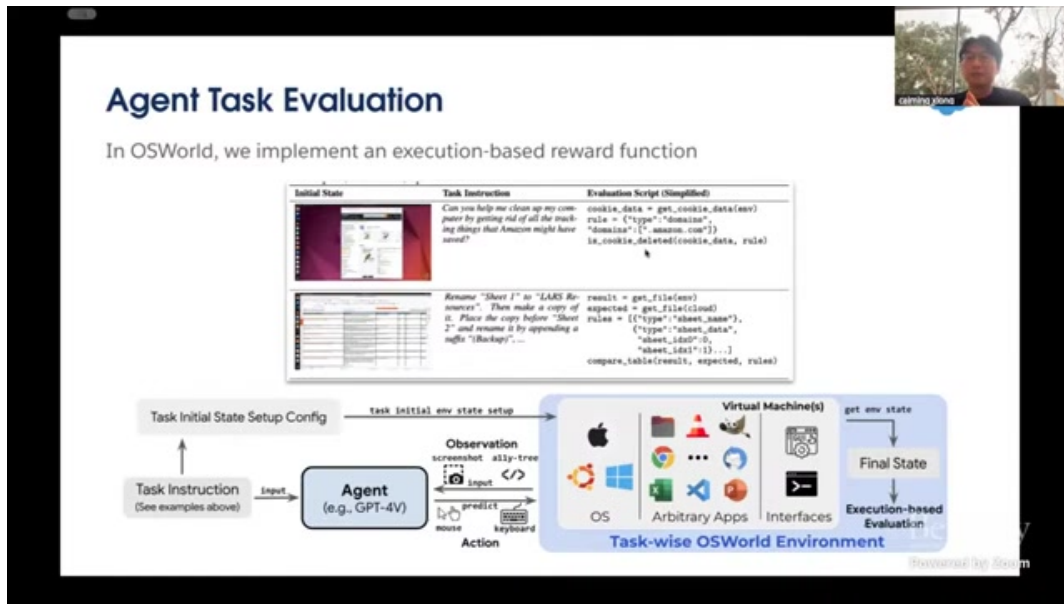


图 4: 视频约 00:19:35: 讲者介绍执行式奖励与部分完成评分逻辑

5.2 部分完成奖励与难度分层

课程强调不应只用二元成败。对复杂任务，部分完成分数能更细粒度反映进步，并显著改善训练信号稀疏问题。实践中应配套难度分层，避免模型长期停在低阶子任务。

奖励函数设计建议

- 将目标拆成关键里程碑，设置分段奖励；
- 对破坏性动作增加惩罚，约束风险行为；
- 对无效重复操作做退火处理，抑制循环轨迹；
- 评估脚本版本化管理，保证可复现性。

5.3 评测体系的鲁棒性要求

评测脚本本身也可能被“攻击”或绕过。课程建议在脚本层加入异常分支、边界检查和反作弊约束，避免模型学习到与任务目标无关的投机策略。

Reward Hacking 是真实风险

如果奖励只覆盖局部状态，Agent 可能学会“骗分”而非“完成任务”。例如通过界面缓存制造假完成状态，或触发脚本漏洞规避关键步骤。评估设计必须先于模型迭代。

5.4 本章小结

Agent 训练的上限很大程度上由评估系统决定。执行式评估、部分奖励和脚本鲁棒性共同决定了模型能否在开放任务上稳定进步。

6 数据瓶颈：从人工标注到教程驱动轨迹生成

6.1 为什么轨迹数据是核心稀缺资源

讲者指出，多步交互轨迹远比单轮指令数据昂贵。每条高质量轨迹通常包含复杂状态切换、工具调用与错误恢复，纯人工采集难以支撑规模化训练。

数据稀缺不在“文本数量”，而在“可执行轨迹质量”

Agent 数据的价值来自三点：任务上下文完整、动作可执行、结果可验证。缺任一项，训练增益都会显著下降。

6.2 教程驱动（Tutorial-driven）数据构造

课程提出从互联网教程中抽取高价值任务模板：先做教程收集与结构化解析，再转为标准化任务描述与步骤候选，最后由 Agent 在环境中执行生成轨迹。

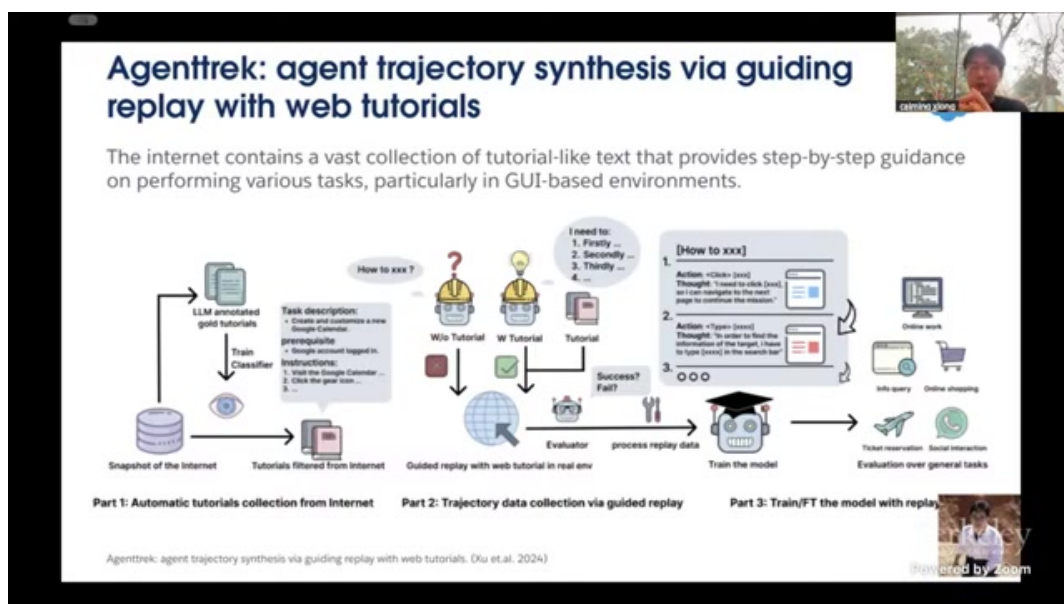


图 5: 视频约 00:44:40: 教程驱动数据构造流程，包含抽取、结构化、执行与筛选

教程驱动方案的优势

- 任务多样性高：覆盖真实用户问题分布；
- 可迁移性好：同类教程可映射为不同环境任务；
- 成本可控：减少纯人工逐步标注；
- 易扩展：可与自动评估脚本打通形成闭环。

6.3 质量控制：从“可运行”到“可学习”

教程文本并不天然等于训练数据。课程强调多级过滤：任务可执行性检查、轨迹一致性检查、错误步骤剔除、低信息样本降权。高噪声数据会显著拉低训练效率。

不要把“采集成功”误判为“数据可用”

很多轨迹虽然执行完毕，但对模型学习几乎无增益，比如重复模板路径、缺少关键决策步骤、动作与观测对齐错误。数据治理必须进入训练主流程，而非离线补丁。

6.4 本章小结

课程给出的数据路线是“自动采集 + 结构化治理 + 脚本验证”。这条路线不追求绝对纯净，而追求在可控成本下持续提升有效样本密度。

7 合成数据与 Action Calling：构建可扩展训练飞轮

7.1 从感知任务到动作任务的迁移

讲者强调，仅靠视觉理解（看懂界面）不足以形成强 Agent，必须把能力迁移到“调用动作”。因此课程引入合成数据管线，把图像问答、OCR、定位、流程推理转化为动作可执行样本。

Action Calling 的本质

Action Calling 不是额外功能，而是把模型输出空间从“描述”切换到“执行”。这要求训练数据同时包含意图、推理路径、动作格式与执行反馈。

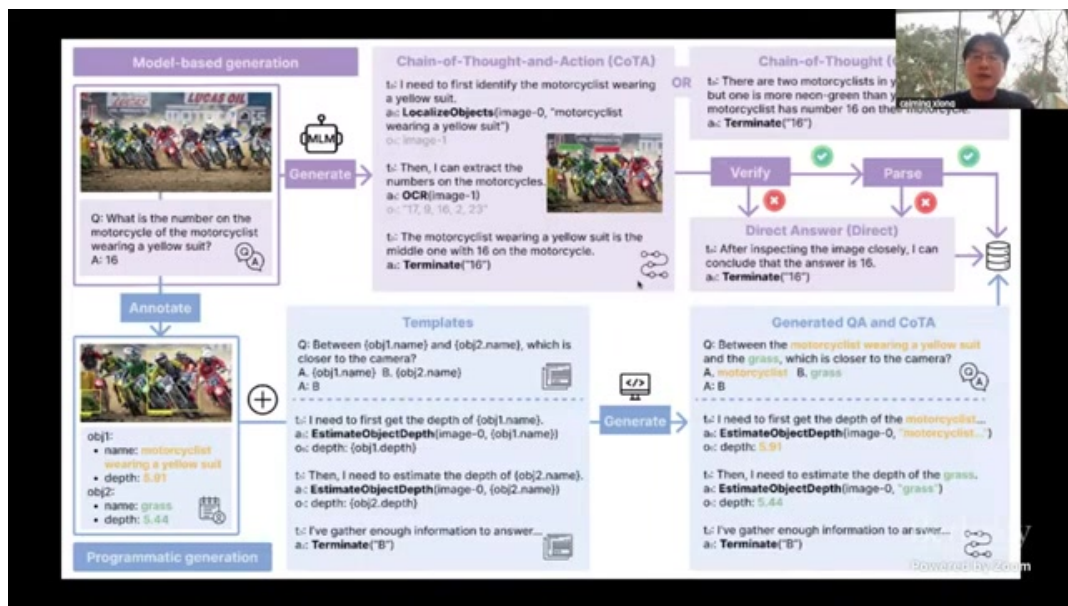


图 6: 视频约 01:02:32：合成数据管线把视觉问题、推理链与动作调用统一到同一训练流程

7.2 典型合成流程

1. 采样任务场景与界面状态；
2. 构造问题与目标（含定位、读取、操作、验证）；
3. 使用模型生成候选推理与动作序列；

4. 借助执行器和评测器筛选高质量样本；
5. 进入分阶段训练并持续回流失败样本。

为什么分阶段训练有效

课程中多次提到“先学看，再学做，再学长期策略”。如果一开始就端到端学习长链路动作，模型更易陷入高方差和局部最优。分阶段可降低优化难度并提升样本利用率。

7.3 合成数据的边界

合成样本虽能扩规模，但分布偏差不可避免。课程建议保持真实轨迹与合成轨迹的比例平衡，并持续做线上任务回归，避免模型只在合成分布内表现良好。

合成数据三大风险

- 语义漂移：目标描述与真实任务意图不一致；
- 动作幻觉：生成看似合理但不可执行的操作；
- 偏差累积：模型生成的数据反过来强化自身错误模式。

7.4 本章小结

合成数据的正确用法是“补充与加速”，不是替代真实交互数据。课程给出的关键经验是：把执行验证作为合成流程的硬约束。

8 模型设计：统一感知、规划与执行的多阶段架构

8.1 现有 GUI Agent 的关键短板

讲者归纳了当前系统的三类问题：视觉 grounding 不稳、长程规划弱、跨平台泛化差。很多模型在短任务上可用，但在高复杂度任务中会快速退化。

三类能力缺口

1. **Grounding**：看到了但定位不准，或目标解析歧义；
2. **Planning**：会执行单步，但缺少全局子目标管理；
3. **Generalization**：换界面、换任务模板后性能骤降。

8.2 课程提出的整合思路

从字幕可见，讲者强调通过统一 AR 风格的生成范式，将视觉理解、推理链（monologue/reasoning trace）和动作生成纳入同一训练接口。这样可减少多模型拼接带来的接口损耗。

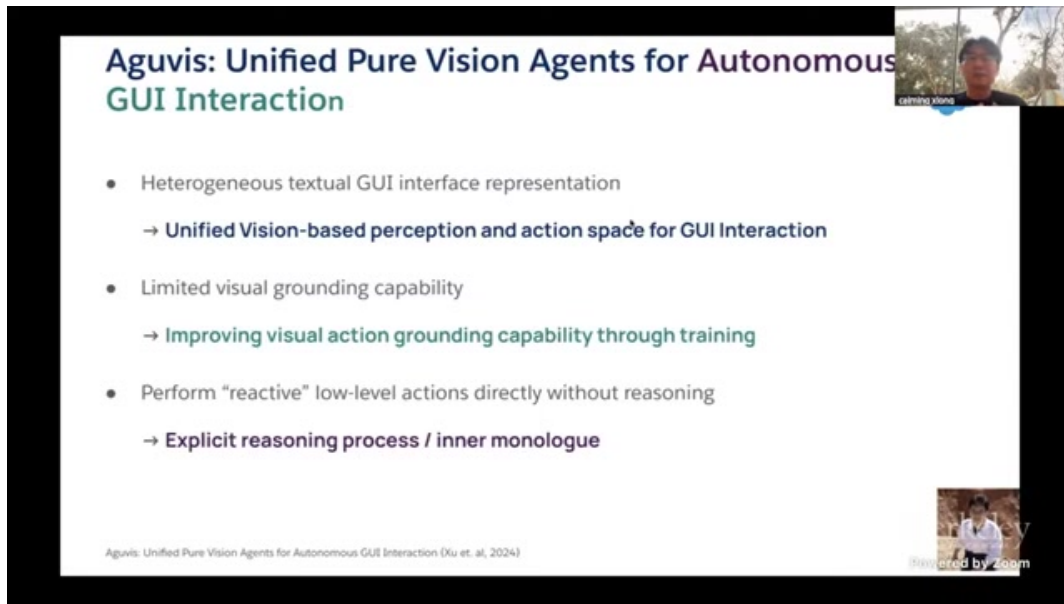


图 7: 视频约 01:16:05: 模型架构强调统一输入输出格式与多阶段训练目标

多阶段训练的工程收益

- 阶段 1: 强化视觉 grounding 与界面语义理解;
- 阶段 2: 引入推理链与规划信号, 提升长程决策;
- 阶段 3: 对齐执行策略与奖励反馈, 优化任务完成率。

8.3 Planner 信号的引入

课程后段反复提到 planner 对难任务收益显著。直观上, planner 不是替代动作模型, 而是提供可解释的中间结构, 让模型在长任务中减少盲目试探。

Planner 不等于“额外冗余”

若任务本身是长链路且状态复杂, 缺 planner 往往意味着更多无效动作与回退。额外规划开销通常小于错误执行成本。是否使用 planner 应按任务复杂度分层决策。

8.4 本章小结

模型架构的关键不在参数规模, 而在“统一接口 + 分阶段训练 + 规划信号”。这三点共同决定了系统在高复杂度任务中的稳定性。

9 实验结果与可操作结论

9.1 结果解读: 趋势比绝对数值更重要

课程展示了在多项 benchmark 上的提升趋势, 尤其在更难任务和在线设置下, 加入规划与更强训练数据治理后效果更稳。即便具体分数随配置变化, 趋势仍非常一致。

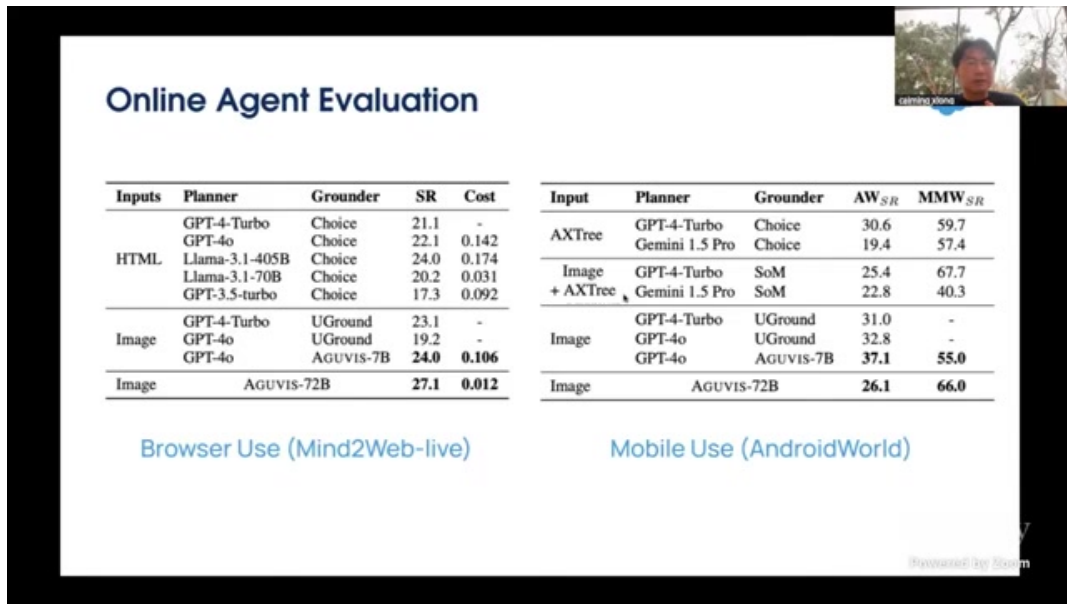


图 8: 视频约 01:24:55: 结果页强调分阶段训练与 planner 对复杂任务成功率的拉升

可复用的三条工程经验

1. 先把评估与环境打稳，再谈模型迭代速度；
2. 数据治理优先级不低于模型升级；
3. 对复杂任务，规划信号通常是“必要条件”而不是“锦上添花”。

9.2 落地指标建议

指标类别	建议指标	使用场景
任务结果	完成率、部分完成率、失败类型分布	评估产品可用性
轨迹质量	平均步数、重复动作率、回退率	诊断规划质量
系统稳定性	重跑一致性、异常中断恢复率	评估生产可靠性
数据闭环	失败样本回流比例、修复后提升幅度	衡量迭代效率

表 2: 从课程结论抽象出的 Agent 工程指标框架

不要只看“总完成率”

单一指标会掩盖问题。比如完成率上升可能来自简单任务占比增加，而不是复杂任务能力提升。课程建议至少按任务难度、任务类型、环境类型做分层统计。

9.3 本章小结

本讲结果最有价值的部分是方法论：环境、评估、数据、模型四者必须同时推进。任何一环短板都会让系统在真实任务上失稳。

10 工程落地路线图：从研究原型到生产系统

10.1 阶段一：可复现实验平台

先构建可重放环境、统一动作协议、标准评测脚本。阶段目标不是追求最高分，而是建立可诊断、可回归、可扩展的实验底座。

10.2 阶段二：数据飞轮

以真实失败任务为核心，持续回流到数据构造与训练流程。教程驱动与合成数据并行推进，但始终保持执行验证约束，防止数据分布漂移。

10.3 阶段三：生产守护

加入权限控制、操作审计、异常回滚、人工接管机制。只有当“可观测性 + 可恢复性”具备，Agent 才能从 demo 进入业务关键链路。

组织层面的协同要求

多模态 Agent 不是单模型团队可以独立完成的项目。需要环境工程、评估工程、数据工程、模型训练和产品安全团队协作，形成统一发布节奏。

一句话落地原则

“先把失败过程看清楚，再去追求更高分数。”这比盲目堆参数或堆样本更能提升真实生产力。

10.4 本章小结

落地路径应遵循“平台先行、数据飞轮、生产守护”。课程的核心价值在于给出了一条可执行、可扩展、可审计的 Agent 工程路线。

11 失败案例与调试手册：把不可用系统变成可迭代系统

11.1 失败类型分层：先分桶再修复

在多模态 Agent 项目里，失败不是例外而是常态。课程隐含的一个关键工程思想是：不要直接盯住“最终失败”，而要先把失败拆成可修复子类。只有当失败类型被分桶，优化动作才会高效。

建议的四层失败分类

1. **感知失败**：没看懂界面、没读到关键文本、目标对象定位错误；
2. **规划失败**：子目标顺序错、漏步骤、重复步骤过多；
3. **执行失败**：动作格式对但上下文不匹配，或接口调用失败；
4. **验证失败**：任务已接近完成，但校验脚本与目标定义不一致。

为什么“失败分桶”会直接影响训练效率

如果所有失败样本都被统一回流，会造成噪声放大。分桶后可以按问题类型定向补数据：感知失败补 grounding 与 OCR 样本，规划失败补长链路推理样本，执行失败补动作约束样本，验证失败补脚本鲁棒性测试样本。

11.2 调试闭环：日志、回放、对照实验

课程内容对应的工程实践可以抽象为三步：第一步抓全量日志，第二步做轨迹回放，第三步做最小对照实验。对照实验的目标不是追求最佳分数，而是确认某个改动是否真实减少了某类失败。

阶段	关键操作	输出物
日志采集	保存观察、动作、环境状态、评测脚本输出	可定位失败发生点
轨迹回放	按时间序列回放 Agent 行为与界面变化	失败原因初步归因
对照实验	单变量替换模型/提示/动作约束/脚本版本	可验证的修复证据

表 3: 多模态 Agent 的最小可行调试流程

避免“一次改很多”

当模型、数据、动作格式、评估脚本同时变化时，几乎无法判断真实增益来源。课程强调工程化迭代，本质上要求每轮实验保持可解释和可回退。

11.3 高频问题实例

- **元素定位偏移**：界面缩放变化导致坐标偏差，建议引入相对定位与对象锚点；
- **长任务退化**：前半段正确、后半段崩溃，常见于上下文压缩和规划遗忘；
- **奖励欺骗**：Agent 学会触发脚本漏洞而非完成任务，需要增强验证脚本鲁棒性；
- **跨应用状态错位**：多个窗口与后台任务并行导致状态不一致，需要更强会话管理。

调试优先级排序

先修复“高频 + 高损失”类失败，再处理低频边缘问题。对生产系统而言，稳定性收益通常大于单次最优表现。

11.4 本章小结

失败调试不是附属工作，而是 Agent 迭代主流程。把失败结构化、可回放、可对照，才能形成真正的训练飞轮。

12 研究议程：下一代多模态 Agent 的三个突破口

12.1 突破口一：统一表示层

课程大量讨论了截图、可访问性树、动作代码和推理链，这些本质上都是不同模态的中间表示。下一阶段的重要方向是建立统一表示层，减少模态切换的信息损耗。

统一表示层要解决的问题

- 如何把视觉对象、语义对象、动作对象映射到同一结构；
- 如何让不同平台（web、desktop、mobile）共享抽象接口；
- 如何支持长链路任务中的状态压缩与可逆恢复。

12.2 突破口二：可验证长程规划

现有模型在复杂任务上仍容易陷入短视策略。课程后段强调 planner 贡献，说明“可验证规划”会成为下一轮竞争焦点：不是只生成计划，而是持续验证计划是否仍与环境状态一致。

规划能力的评测应从“有无”进化到“质量”

未来评测不仅要问模型能否给出 plan，还要评估 plan 的可执行性、可更新性和错误恢复效率。没有这些指标，planner 容易沦为装饰性中间文本。

12.3 突破口三：安全与治理原生

当 Agent 开始操作真实账户和业务系统，安全不可能后置。权限边界、审计日志、异常回滚和人类接管机制必须成为模型训练和部署时的原生约束，而非上线前补丁。

Agent 的最大风险不只是“做不到”，而是“做错且不可追溯”

如果系统缺少可审计链路，事故发生后无法复盘责任边界。课程虽然聚焦研究，但其环境与评估思想已经为治理原生提供了实践基础。

12.4 未来 12 个月可执行任务清单

方向	建议任务	预期产出
统一表示	构建跨平台对象-动作中间层	降低迁移成本与动作错误率
规划优化	引入在线计划校验与重规划机制	提升长任务完成率
数据飞轮	失败样本自动分桶并回流训练	降低高频失败复现率
安全治理	建立权限模板、审计与回滚策略	支撑生产可控部署

表 4: 从课程方法论落到团队执行层的路线图

12.5 本章小结

下一代多模态 Agent 的竞争，不会只看模型参数规模，而会看谁能把统一表示、长程规划和治理机制做成可复用工程能力。

13 总结与延伸

13.1 全讲总结表

主题	核心观点	工程动作
问题定义	Agent 目标从答题转向真实任务闭环	建立环境交互评测而非仅离线评分
Benchmark	OS-World 补齐真实计算机场景缺口	任务脚本化、环境可重放、结果可验证
数据体系	轨迹数据是主要瓶颈	教程驱动采集 + 合成增强 + 质量治理
模型体系	统一感知/规划/执行并分阶段训练	引入规划信号，强化 grounding 与长程决策
落地策略	生产可用性依赖可观测与可恢复	增加审计、回滚、权限和人工接管机制

表 5: 课程结论的工程化抽象

13.2 进一步阅读

- OS-World 论文与项目文档（真实计算机环境 Agent benchmark）
- WebArena / VisualWebArena / Mind2Web（Web Agent 评测演进）
- GUI grounding 与 action modeling 相关工作（视觉定位到动作执行）

- 多模态 Agent 数据合成与轨迹筛选论文（可扩展训练数据管线）
- Agent 安全与审计实践（权限、回滚、可观测性）

13.3 延伸问题

- 如何设计跨应用任务的统一中间表示，减少平台耦合？
- 在在线生产环境中，如何平衡自动化效率与安全边界？
- 当任务目标不可完全脚本化时，评估系统如何与人工反馈协同？